分享前准备

请提前在个人电脑上准备：

1. 安装 Node.js LTS（包含 npm/npx）和最新版 Chrome；本案例建议 Node.js 24 LTS（24.12.0 及以上），也支持 22 LTS（22.18.0 及以上）。
2. 后端案例需要 **JDK 21 LTS**：下载 OpenJDK 21 / Eclipse Temurin，选择版本21、JDK、对应操作系统和CPU架构，不要只安装JRE。
3. 按 Codex CLI 官方安装说明安装，完成登录，确认终端可以执行 `codex`。
4. 安装 VS Code，并安装 Codex 插件，完成登录，确认可打开 codex 会话。
5. 运行下面的环境准备脚本，等到 Chromium 启动检查、Chrome DevTools MCP 安装及 Codex 配置检查成功，并输出专用 conda 环境名称。
6. 用 VS Code 打开整个分享目录。
7. 与本次分享不直接相关，后续实际项目，推荐安装 Obsidian 打开和管理 md 知识库。
8. 且推荐安装 RTK 插件，可以节省 token。

JDK下载选择：Windows通常选择x64的MSI安装包；macOS按Intel选择x64、Apple Silicon选择aarch64，可用PKG安装包；Linux按x64/aarch64选择安装包或压缩包。安装后将 `JAVA_HOME` 指向JDK21根目录，将该目录的 `bin` 加入PATH；Windows安装器可勾选相应选项。已有多个JDK时，确认当前终端的java和javac都来自JDK21。环境准备脚本不负责安装Node.js或JDK。

重新打开终端后检查：

```
node --version
npm --version
java -version
javac -version
```

环境准备

所有命令均在分享目录执行。首次安装需要联网。

Windows PowerShell 或 CMD：

```
.\prepare-environment.bat
```

在 CMD 中运行时，脚本完成后当前终端已经激活；从 PowerShell 执行 bat 时无法改变父终端环境，可进入已激活的 CMD 继续工作：

```
cmd /k .environment\activate.bat
```

macOS / Linux：

```
bash ./prepare-environment.sh
source .environment/activate.sh
```

脚本会检查已有 Python、venv、uv 和 conda 的版本及可用情况，优先复用 conda；没有 conda 时，安装当前用户的 Miniconda，并将 conda 加入用户 PATH，后续新终端可直接使用。

分享专用 conda 环境名为 `codex-sharing-<目录标识>`，根据分享目录生成并重复复用，避免与其他目录共用环境；名称和实际位置记录在 `.environment/environment.json`。Playwright、pytest-playwright 和 Chromium 浏览器文件均保存在这个环境中。脚本重复执行会检查并补齐缺失内容，中断后重跑同一脚本即可继续；下载文件会保留，下载服务支持时可续传。默认不升级已有包。

脚本同时安装 Chrome DevTools MCP 到 `.environment/cache/chrome-devtools-mcp/`，首次安装锁定实际版本，后续复用可用安装；中断后重跑继续补齐。通过 Codex CLI 注册 `chrome-devtools`，启用 `--autoConnect`，重复运行配置相同时跳过。已有同名服务器配置不匹配时更新该配置，其他 MCP 保留。缺少可用 Node.js LTS/npm 或 Codex CLI 时明确报错，请补齐后重跑。

系统	执行入口与区别
macOS	Bash 脚本；Miniconda 按 Intel / Apple Silicon 选择；MCP 使用实际 Node 和 JS 入口路径
Linux	Bash 脚本；Miniconda 按 x86_64 / ARM64 选择；Playwright 缺失系统库按提示补齐；MCP 使用实际 Node 和 JS 入口路径
Windows	BAT 调用 PowerShell；Miniconda 使用 Windows 安装器；MCP 用 node.exe 直接运行 JS，npm/Codex 的 CMD 包装也转换成 JS 入口，避免含空格路径的包装命令问题

准备成功后重新打开 Codex CLI，或重启 VS Code 中的 Codex 扩展，用 `/mcp` 查看加载状态。采集前打开 Chrome，在 `chrome://inspect/#remote-debugging` 启用远程调试并保持目标网页标签打开；出现本次连接提示时允许连接。脚本只校验安装与配置，不操作 Chrome，也不把环境准备成功当作连接或采集成功。

若浏览器启动提示缺少系统库，按提示补齐操作系统依赖，再重跑；浏览器程序保存在环境里，系统库仍由操作系统提供。Windows 和其他系统的虚拟环境不能互相复制使用，应各自运行准备脚本。

分享目标

通过常用命令介绍和案例带教，帮助产品与技术同学掌握：

- Codex CLI 的启动、会话恢复、模型与思考等级选择、goal 和 remote-control。

- AGENTS、Memory、Plugin、SKILL 与子 Agent 的作用，以及用工作约定减少重复提示。
- 从需求 prompt 落地跨平台脚本，把网站采集和 UI 测试等重复任务做成通用 SKILL。
- 用结构化 Markdown 知识库串联参考采集、产品与原型、系统与测试设计、前后端实现和本地部署，最后用 Playwright 验证。
- 区分设计、Mock 演示、真实服务联调与测试通过，用实际操作和证据判断完成情况。

后续使用

新终端先按上述方式激活专用 conda 环境，Python 任务都用该环境，新增依赖安装到同一环境：

```
python -m pip install 包名
codex
```

分享入口：00_分享入口.md。案例生成物放在 `output/` 并加入 Git 忽略；conda 环境保存在 conda 的环境目录，本机激活文件、环境记录和下载缓存也不提交 Git。

官方说明：Miniconda 安装、Playwright Python 安装、浏览器文件位置。

MCP 说明：Chrome DevTools MCP、Codex MCP 配置。